

Fundamentals of Software Development

# Assessment Task 1 – Part 1

Cmp1\_07

Group 6

Sheyda Niaparast - 26496772

Huilin Zeng - 26310336

Chengfeng Yang - 26642759

Sihan Yin - 26151769

Spring Semester

2026

## Table of Contents

|                                                  |           |
|--------------------------------------------------|-----------|
| <b>1. User-Story Backlog.....</b>                | <b>3</b>  |
| <b>1.1. Narrative user stories .....</b>         | <b>3</b>  |
| <b>1.2. Requirements table (backlog) .....</b>   | <b>4</b>  |
| <b>2. UML Use-Case Diagram .....</b>             | <b>6</b>  |
| <b>3. UML Class Diagram.....</b>                 | <b>8</b>  |
| <b>3.1. Classes.....</b>                         | <b>9</b>  |
| <b>3.2. Fields and visibility .....</b>          | <b>10</b> |
| <b>3.3. Methods and visibility .....</b>         | <b>11</b> |
| <b>3.4. Relationships and multiplicity .....</b> | <b>13</b> |

# 1. User-Story Backlog

The university requirements were transformed into simple, generic user stories. Each story was first expressed in a narrative form “As a [user], I want to [do something] so that [result happens],” then organized into the required backlog columns: ID, User, Action, Result / Outcome, and Function. Each story captures one main action and its expected result. The stories focus primarily on user goals and meaningful system behavior. Every story has a unique three-digit ID, and stories belonging to the same feature share the same hundreds digit: 1xx Login, 2xx Register, 3xx Enroll, 4xx Drop, 5xx View, 6xx Change Password, 7xx Admin, and 8xx File I/O. “System” appears in the User column where the action is performed automatically by the application.

## 1.1. Narrative user stories

### 1xx — Login

- *As a student, I want to log in with my email and password so that I can access the student subsystem.*
- *As the system, I want to validate the login details so that invalid credentials can be rejected.*

### 2xx — Register

- *As a student (unregistered), I want to register with my name, email and password so that I become a student who can use the system.*
- *As the system, I want to validate the registration details and assign a unique student ID so that a valid student record can be created.*

### 3xx — Enroll

- *As a student, I want to enroll in a subject so that it is added to my enrolment list.*
- *As the system, I want to create a subject with a unique ID, random mark and grade so that the enrolment information is complete.*
- *As the system, I want to check the number of enrolled subjects so that a student can't enroll in more than four subjects.*

### 4xx — Drop

- *As a student, I want to drop a subject so that it is removed from my enrolment list.*

### 5xx — View

- *As a student, I want to view my enrolment list so that I can see my subjects, marks and grades.*
- *As an admin, I want to view all registered students so that I can see every student on file.*

### 6xx — Change Password

- *As a student, I want to change my password so that my login credentials stay up to date.*
- *As the system, I want to validate the new password so that only a password meeting the required pattern is accepted.*

### 7xx — Admin

- *As an admin, I want to view, organize and categorize all students so that I can monitor their results.*
- *As an admin, I want to remove a student or clear all records so that I can organized the student data as I see fit.*

### 8xx — File I/O

- *As the system, I want to save and load student data so that information persists between sessions.*

## 1.2. Requirements table (backlog)

| ID                    | User                     | Action                                                                                                   | Result / Outcome                                                  | Function               |
|-----------------------|--------------------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|------------------------|
| <b>1xx — Login</b>    |                          |                                                                                                          |                                                                   |                        |
| 101                   | Student (R) <sup>1</sup> | log into the application if given valid credentials                                                      | Student is taken to the student subsystem                         | login()                |
| 102                   | System                   | match the entered email and password with the ones on file                                               | a registered student is authenticated                             | match_credentials()    |
| 103                   | System                   | verify the entered email and password against the required patterns                                      | malformed input is rejected before it is checked against the file | validate_credentials() |
| 104                   | System                   | show an error message when the credentials do not match                                                  | the student is told the login failed and can retry                | show_login_error()     |
| <b>2xx — Register</b> |                          |                                                                                                          |                                                                   |                        |
| 201                   | Student (U) <sup>2</sup> | register by entering a name, an email and a password                                                     | a new student account is created                                  | Register_account()     |
| 202                   | System                   | validate the email against pre-defined patterns (e.g. emails should end with the domain @university.com) | only valid university emails are accepted                         | validate_email()       |
| 203                   | System                   | validate the password (upper-case start, at least five letters, followed by three or more digits)        | only passwords meeting the criteria are accepted                  | validate_password()    |
| 204                   | System                   | generate and assign a unique six-digit student ID (zero-padded, 1–999,999)                               | each student is uniquely identified                               | generate_ID()          |
| 205                   | System                   | show an error message when the Register details are invalid                                              | the visitor is prompted to re-enter valid details                 | show_register_error()  |
| <b>3xx — Enroll</b>   |                          |                                                                                                          |                                                                   |                        |
| 301                   | Student (R)              | request to enroll in a subject                                                                           | the enrolment request is submitted                                | enrol_subject()        |
| 302                   | System                   | auto-generate a random mark between 25 and 100 for the enrolled subject                                  | the subject is assigned a mark                                    | generate_mark()        |
| 303                   | System                   | calculate the subject grade from its mark (Z / P / C / D / HD on the UTS scale)                          | the subject is assigned a grade                                   | calculate_grade()      |
| 304                   | System                   | reject enrolment when the list already holds four subjects                                               | the student is notified the four-subject limit is reached         | check_subject_limit()  |
| 305                   | System                   | generate a unique three-digit ID for the subject (001-999)                                               | a subject ID between 001 and 999 is assigned                      | generate_subject_ID()  |
| 306                   | System                   | add the generated subject to the student's enrolment list                                                | the new subject is included in the student's enrolments           | add_subject()          |

<sup>1</sup> (R) stands for Registered

<sup>2</sup> (U) stands for Unregistered

| ID                           | User        | Action                                                                          | Result / Outcome                                                | Function                     |
|------------------------------|-------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------|------------------------------|
| 307                          | System      | Show an enrolment-limit error when the student already has four subjects        | The student is notified that no more subjects can be added      | show_enrolment_limit_error() |
| <b>4xx — Drop</b>            |             |                                                                                 |                                                                 |                              |
| 401                          | Student (R) | remove (drop) a subject from the enrolment list                                 | the selected subject is removed from the list                   | remove_subject()             |
| <b>5xx — View</b>            |             |                                                                                 |                                                                 |                              |
| 501                          | Student (R) | view the current enrolment list                                                 | all enrolled subjects with their marks and grades are displayed | view_enrolment()             |
| 502                          | Admin       | view all registered students                                                    | every student on file is displayed                              | view_all_students()          |
| <b>6xx — Change Password</b> |             |                                                                                 |                                                                 |                              |
| 601                          | System      | validate the new password against the required pattern                          | only a valid new password is accepted                           | validate_password()          |
| 602                          | Student (R) | Update the student's password                                                   | The valid new password replaces the previous password           | update_password()            |
| <b>7xx — Admin</b>           |             |                                                                                 |                                                                 |                              |
| 701                          | Admin       | access the admin subsystem without registering                                  | existing staff can manage students directly                     | admin_access()               |
| 702                          | Admin       | organize and view students grouped by grade                                     | students are displayed grouped by grade                         | group_by_grade()             |
| 703                          | Admin       | categorize students into PASS/FAIL by mark (mark < 50 → FAIL; mark ≥ 50 → PASS) | students are classified by outcome                              | categorise_pass_fail()       |
| 704                          | Admin       | remove an individual student                                                    | a single student record is deleted                              | remove_student()             |
| 705                          | Admin       | clear all student records                                                       | the students.data file is emptied                               | clear_all_students()         |
| <b>8xx — File I/O</b>        |             |                                                                                 |                                                                 |                              |
| 801                          | System      | save registered students and their enrolment details to students.data           | student data persists between sessions                          | save_to_file()               |
| 802                          | System      | load student data from students.data on start-up                                | the data is available to the student and admin subsystems       | load_from_file()             |

Table 1. User-story backlog

## 2. UML Use-Case Diagram

Figure 1 presents the CLIUniApp, a command-line application for student enrolment and administration within one semester. Actors appear outside the system boundary, and solid lines connect them to the use cases they initiate. An <<include>> arrow points to required supporting behavior. An <<extend>> arrow points from additional behavior to its base use case as it represents behaviors that occur under specific conditions. These relationships describe required and conditional behaviors rather than the sequence of user actions.

The CLIUniApp diagram identifies three actors: Unregistered Student, Registered Student, and Admin. The “Unregistered Student” aims to create an account through “Register account”. The student provides a name, email and password. Registration has <<include>> relationships with “Validate email”, “Validate password” and “Generate ID”. The first two check the input against the required patterns. Generate ID assigns a unique six-digit student ID between 000001 and 999999 after validation succeeds. “Show registration error” extends the register function when registration details are invalid. The student must correct the details before an account can be created.

The “Registered Student” aims to access the student subsystem and manage personal enrolments. “Log in” includes two distinct checks: “Validate credentials” checks input formats, while “Match credentials” compares validly formatted input with stored records. The checks correspond to backlog items 103 and 102 respectively. “Show login error” extends the log in function when the credentials do not match. Successful login is a precondition for enrolment, subject removal, viewing the enrolment list and password updates.

“Enrol” in a subject includes “Check subject limit” (304). The separate error message corresponds to backlog item 307. If the student already has four subjects and requests another, “Show enrolment limit error” extends enrolment and no subject is added. A permitted enrolment includes “Generate subject ID”, “Generate mark” and “Add subject”. The system creates a unique three-digit subject ID, assigns a random mark from 25 to 100 and adds the subject to the student’s list. The student does not choose a specific subject. In this diagram, “Generate mark” also includes “Calculate grade”, which assigns Z, P, C, D or HD based on the generated mark.

“Remove a subject” allows the student to drop an existing enrolment. “View current enrollment list” displays subjects, marks and grades. “Update password” replaces the existing password after validating it. Its <<include>> relationship with “Validate password” represents the same password check used during registration.

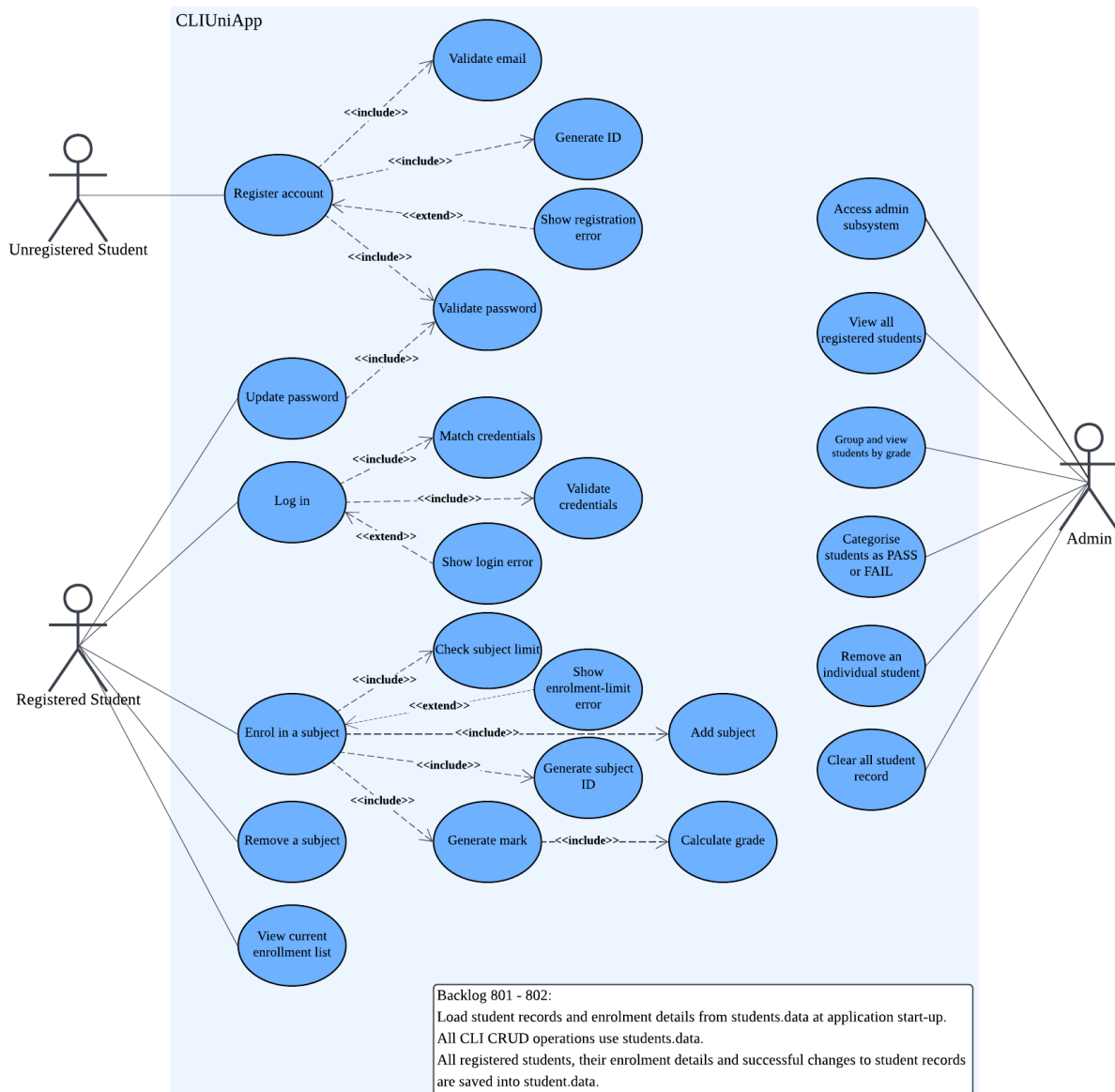


Figure 1. UML Use-Case diagram

The “Admin” represents existing university staff and aims to manage student records through Access admin subsystem. Access admin subsystem does not require registration. Administrators can view all registered students, group students by grade and categories them as PASS or FAIL. Marks below 50 indicate FAIL and marks of at least 50 indicate PASS. “Remove an individual student” deletes one record, while “Clear all student record” deletes every record. Solid associations connect the “Admin” directly to these management functions.

The diagram’s note covers backlog items 801–802. The application loads student.data at start-up, uses this file for all CLI CRUD operations and saves successful changes. File saving also applies after deletions and password changes. These are shared storage requirements, not separate actors. They support consistent student and administrator access to the same records.

### 3. UML Class Diagram

Figure 2 presents the UML class diagram for CLIUniApp. The diagram demonstrates the classes required to fulfill the requirements backlog, along with their fields, methods, member visibility, and the relationships and multiplicities between them. Member visibility follows standard UML notation: a plus sign (+) marks public members, a minus sign (−) marks private members, and a hash (#) marks protected members. Every field is shown with its data type and every method with its parameter types and return type, and each method name corresponds directly to a function in the backlog so that the design remains traceable to the requirements. The classes are also grouped into two layers: the CLIUniApp control layer, which holds the controller that drives the command-line interaction, and the model, persistence and utility layer, which holds the domain classes, the file-backed store and the validation helper. The two containers group the classes by responsibility and are not UML elements.

The diagram uses five UML relationship types. Generalisation (a solid line with a hollow triangle) shows that one class is a kind of another. Composition (a solid line with a filled diamond) shows a strong whole–part ownership in which the part cannot exist without its whole. Aggregation (a solid line with a hollow diamond) shows a weaker whole–part relationship in which the part can exist independently. An association (a plain solid line) shows that one class holds a reference to another and works through it. A dependency (a dashed arrow) shows that one class uses the operations of another without owning an instance of it. The numbers at each end of an association state its multiplicity.

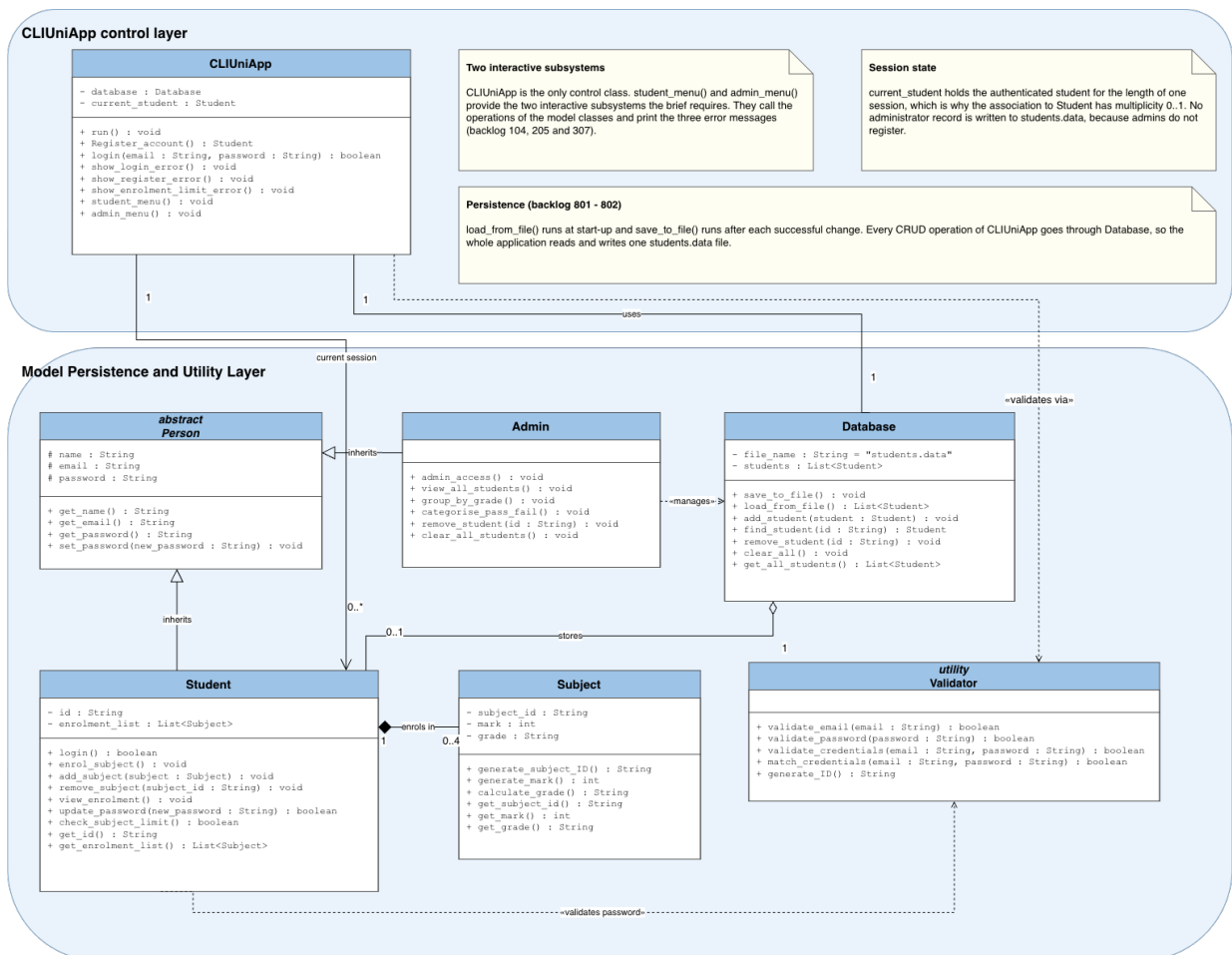


Figure 2. UML class diagram for CLIUniApp

### 3.1. Classes

- **Person (abstract).** Person is an abstract superclass that captures the attributes shared by every human actor in the system: name, email and password. These fields are declared protected (#) so that the Student and Admin subclasses can reuse them while they stay hidden from unrelated classes. Person is abstract because the system never instantiates a bare person — only concrete students and admins exist. It exposes public accessors `get_name()`, `get_email()` and `get_password()`, and a `set_password()` mutator used when a password is changed.
- **Student (extends Person).** Student represents a registered student. It adds a private id (the zero-padded six-digit identifier assigned at registration) and a private enrolment list holding the student's Subject objects. Its public methods realise the student related functions in the backlog: `login()` (101), `enrol_subject()` (301), `add_subject()` (306), `remove_subject()` (401), `view_enrolment()` (501), `update_password()` (602) and `check_subject_limit()` (304), together with the accessors `get_id()` and `get_enrolment_list()`. Because a student may hold at most four subjects, the composition to Subject is bounded by 0..4 and `check_subject_limit()` enforces the rule before a new subject is added.
- **Admin (extends Person).** Admin represents existing university staff, who do not register and therefore add no persistent fields of their own; the identity fields are inherited from Person. Its public methods realise the admin related functions in the backlog: `admin_access()` (701), `view_all_students()` (502), `group_by_grade()` (702), `categorise_pass_fail()` (703), `remove_student()` (704) and `clear_all_students()` (705). Because these operations act on the entire student collection rather than on the admin's own state, Admin depends on Database.
- **Subject.** Subject describes a single enrolment. It holds a private subject id (the unique three-digit ID, 001–999), a private mark (a random integer from 25 to 100) and a private grade (Z, P, C, D or HD on the UTS scale). It owns the logic that populates these fields, `generate_subject_ID()` (305), `generate_mark()` (302) and `calculate_grade()` (303), with the accessors `get_subject_id()`, `get_mark()` and `get_grade()`. A Subject has no meaning independent of the student who enrolled in it, which is why the relationship is modelled as composition.
- **Validator (utility).** Validator is a stateless helper class that concentrates the pattern-checking rules, so they are defined once and reused across registration, login and password changes. Its public methods are `validate_email()` (202), `validate_password()` (203, 601), `validate_credentials()` (103), `match_credentials()` (102) and `generate_ID()` (204). It holds no fields; the «utility» feature signals that its operations are used without holding an object of the class. Both CLIUniApp and Student depend on it.
- **Database.** Database encapsulates all persistence to the students.data file and owns the in-memory collection of Student objects. It holds a private file name constant ("students.data") and a private students list. Its public methods provide the file-I/O backlog and the CRUD operations both subsystems need: `save_to_file()` (801), `load_from_file()` (802), `add_student()`, `find_student()`, `remove_student()`, `clear_all()` and `get_all_students()`. Because the stored student records exist independently of any single run of the application, Database's relationship to Student is modelled as aggregation rather than composition.
- **CLIUniApp.** CLIUniApp is the application controller and entry point that coordinates the other classes. It owns a Database (composition, the file-backed store is created and managed by the application) and keeps a reference to the current student for the active session. Its methods drive the command-line flow: `run()` is the main loop; `Register_account()` (201) creates a new student after validation; `login()` (101) authenticates a returning student; `show_login_error()` (104), `show_register_error()` (205) and `show_enrolment_limit_error()` (307) present the three required error messages; and `student_menu()` and `admin_menu()` route the user into the correct subsystem.

### 3.2. Fields and visibility

Table 2 lists each class's fields with their data types and visibility. All fields are private (–) or protected (#) so that state is encapsulated, read or changed only through the classes' public methods. The final column justifies the visibility chosen for each group of fields.

| Class            | Field                                        | Type          | Visibility  | Justification for visibility                                                                                                                                                                                      |
|------------------|----------------------------------------------|---------------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Person</b>    | name                                         | String        | # protected | Protected (#) rather than private so the Student and Admin subclasses inherit these shared identity fields, while classes outside the hierarchy cannot reach them.                                                |
|                  | email                                        | String        | # protected |                                                                                                                                                                                                                   |
|                  | password                                     | String        | # protected |                                                                                                                                                                                                                   |
| <b>Student</b>   | id                                           | String        | – private   | Private (–) to protect state the class must keep valid: id is generated once and must never be reassigned from outside, and enrolment_list is guarded by check_subject_limit() so it never exceeds four subjects. |
|                  | enrolment_list                               | List<Subject> | – private   |                                                                                                                                                                                                                   |
| <b>Admin</b>     | (inherits name, email, password from Person) | —             | —           | No fields of its own; the identity fields are inherited from Person (protected).                                                                                                                                  |
| <b>Subject</b>   | subject_id                                   | String        | – private   | Private (–) so the values stay consistent; mark must remain 25–100 and determines grade and are set only by the class's own generate_ and calculate_ methods.                                                     |
|                  | mark                                         | int           | – private   |                                                                                                                                                                                                                   |
|                  | grade                                        | String        | – private   |                                                                                                                                                                                                                   |
| <b>Validator</b> | (no fields — utility class)                  | —             | —           | No fields; a stateless «utility» class whose operations are used without holding an object.                                                                                                                       |
| <b>Database</b>  | file_name = "students.data"                  | String        | – private   | Private (–) to encapsulate persistence: file_name is fixed and the student collection is changed only through the class's CRUD and file-I/O methods.                                                              |
|                  | students                                     | List<Student> | – private   |                                                                                                                                                                                                                   |
| <b>CLIUniApp</b> | database                                     | Database      | – private   | Private (–) so the controller alone owns its data store and current-session state, exposed only through its public flow and menu methods.                                                                         |
|                  | current_student                              | Student       | – private   |                                                                                                                                                                                                                   |

Table 2. Class fields, types, visibility, and justification for each visibility choice

### 3.3. Methods and visibility

Table 3 lists the methods of each class with their parameter types, return types and visibility, and maps each to its backlog item to confirm that every required function is implemented by exactly one class. All methods are public (+); the final column justifies that choice for each class's group of methods.

| Class          | Method (signature)                     | Return        | Vis. | Backlog ID | Justification for visibility                                                                                                                                                           |
|----------------|----------------------------------------|---------------|------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Person</b>  | get_name()                             | String        | +    | —          | Public (+) so subclasses and collaborators can read the identity data and change a password only through these accessors and mutator, never by touching the protected fields directly. |
|                | get_email()                            | String        | +    | —          |                                                                                                                                                                                        |
|                | get_password()                         | String        | +    | —          |                                                                                                                                                                                        |
|                | set_password(new_password : String)    | void          | +    | —          |                                                                                                                                                                                        |
| <b>Student</b> | login()                                | boolean       | +    | 101        | Public (+) because these are the student subsystem's operations that CLIUniApp invokes after login; the accessors give controlled read access to the private id and enrolment list.    |
|                | enrol_subject()                        | void          | +    | 301        |                                                                                                                                                                                        |
|                | add_subject(subject : Subject)         | void          | +    | 306        |                                                                                                                                                                                        |
|                | remove_subject(subject_id : String)    | void          | +    | 401        |                                                                                                                                                                                        |
|                | view_enrolment()                       | void          | +    | 501        |                                                                                                                                                                                        |
|                | update_password(new_password : String) | boolean       | +    | 602        |                                                                                                                                                                                        |
|                | check_subject_limit()                  | boolean       | +    | 304        |                                                                                                                                                                                        |
|                | get_id()                               | String        | +    | —          |                                                                                                                                                                                        |
|                | get_enrolment_list()                   | List<Subject> | +    | —          |                                                                                                                                                                                        |
| <b>Admin</b>   | admin_access()                         | void          | +    | 701        |                                                                                                                                                                                        |

| Class            | Method (signature)                                      | Return  | Vis. | Backlog ID | Justification for visibility                                                                                                                                                                       |
|------------------|---------------------------------------------------------|---------|------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                  | view_all_students()                                     | void    | +    | 502        | Public (+) as they are the admin subsystem's operations, called by CLIUniApp when an admin is using the system.                                                                                    |
|                  | group_by_grade()                                        | void    | +    | 702        |                                                                                                                                                                                                    |
|                  | categorise_pass_fail()                                  | void    | +    | 703        |                                                                                                                                                                                                    |
|                  | remove_student(id : String)                             | void    | +    | 704        |                                                                                                                                                                                                    |
|                  | clear_all_students()                                    | void    | +    | 705        |                                                                                                                                                                                                    |
| <b>Subject</b>   | generate_subject_ID()                                   | String  | +    | 305        | Public (+) so the owning Student and the controller can build and read a subject; the generate_ and calculate_ methods set the private fields internally, and the accessors expose them read-only. |
|                  | generate_mark()                                         | int     | +    | 302        |                                                                                                                                                                                                    |
|                  | calculate_grade()                                       | String  | +    | 303        |                                                                                                                                                                                                    |
|                  | get_subject_id()                                        | String  | +    | —          |                                                                                                                                                                                                    |
|                  | get_mark()                                              | int     | +    | —          |                                                                                                                                                                                                    |
|                  | get_grade()                                             | String  | +    | —          |                                                                                                                                                                                                    |
| <b>Validator</b> | validate_email(email : String)                          | boolean | +    | 202        | Public (+) so any class can reuse the shared validation and ID-generation rules; being a «utility», the methods are called without instantiating the class.                                        |
|                  | validate_password(password : String)                    | boolean | +    | 203, 601   |                                                                                                                                                                                                    |
|                  | validate_credentials(email : String, password : String) | boolean | +    | 103        |                                                                                                                                                                                                    |
|                  | match_credentials(email : String, password : String)    | boolean | +    | 102        |                                                                                                                                                                                                    |
|                  | generate_ID()                                           | String  | +    | 204        |                                                                                                                                                                                                    |

| Class            | Method (signature)                       | Return        | Vis. | Backlog ID | Justification for visibility                                                                                                                                                                         |
|------------------|------------------------------------------|---------------|------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Database</b>  | save_to_file()                           | void          | +    | 801        | Public (+) because they are the only sanctioned way to read or modify persisted data, all file access and CRUD passes through this interface, keeping the file name and students collection private. |
|                  | load_from_file()                         | List<Student> | +    | 802        |                                                                                                                                                                                                      |
|                  | add_student(student : Student)           | void          | +    | —          |                                                                                                                                                                                                      |
|                  | find_student(id : String)                | Student       | +    | —          |                                                                                                                                                                                                      |
|                  | remove_student(id : String)              | void          | +    | 704        |                                                                                                                                                                                                      |
|                  | clear_all()                              | void          | +    | 705        |                                                                                                                                                                                                      |
|                  | get_all_students()                       | List<Student> | +    | —          |                                                                                                                                                                                                      |
| <b>CLIUniApp</b> | run()                                    | void          | +    | —          | Public (+) as the application's entry points and menu flow, invoked to run the program, register or authenticate a user, display errors, and route to the correct subsystem.                         |
|                  | Register_account()                       | Student       | +    | 201        |                                                                                                                                                                                                      |
|                  | login(email : String, password : String) | boolean       | +    | 101        |                                                                                                                                                                                                      |
|                  | show_login_error()                       | void          | +    | 104        |                                                                                                                                                                                                      |
|                  | show_register_error()                    | void          | +    | 205        |                                                                                                                                                                                                      |
|                  | show_enrolment_limit_error()             | void          | +    | 307        |                                                                                                                                                                                                      |
|                  | student_menu()                           | void          | +    | —          |                                                                                                                                                                                                      |
|                  | admin_menu()                             | void          | +    | —          |                                                                                                                                                                                                      |

Table 3. Class methods, return types, visibility, backlog mapping, and justification for each visibility choice

### 3.4. Relationships and multiplicity

Table 4 lists every relationship in the diagram with its type and multiplicity. The multiplicity column reads from the source class to the target class.

| From             | To        | Relationship   | Multiplicity | Description                                                                                             |
|------------------|-----------|----------------|--------------|---------------------------------------------------------------------------------------------------------|
| <b>Student</b>   | Person    | Generalisation | —            | A student is a kind of person and inherits its identity fields.                                         |
| <b>Admin</b>     | Person    | Generalisation | —            | An admin is a kind of person and inherits its identity fields.                                          |
| <b>Student</b>   | Subject   | Composition    | 1 → 0..4     | A student owns up to four subjects; each subject belongs to exactly one student and is removed with it. |
| <b>CLIUniApp</b> | Database  | Composition    | 1 → 1        | The application creates and owns a single file-backed data store.                                       |
| <b>Database</b>  | Student   | Aggregation    | 1 → 0..*     | The store holds any number of student records that persist independently of a session.                  |
| <b>CLIUniApp</b> | Student   | Association    | 1 → 0..1     | At most one student is logged in during a session (the current student).                                |
| <b>CLIUniApp</b> | Validator | Dependency     | —            | Uses the validation and ID-generation operations during register and login.                             |
| <b>Student</b>   | Validator | Dependency     | —            | Uses password validation when updating a password.                                                      |
| <b>Admin</b>     | Database  | Dependency     | —            | Operates on the whole student collection to view, categorise and remove records.                        |

*Table 4. Class relationships and multiplicities*